

ISW 2° Parcial

Framework para Escalar Scrum

Dado que Scrum limita el tamaño de los equipos a 7 ± 2 integrantes, surge la necesidad de escalar este framework para poder gestionar productos de mayor envergadura, ya que las prácticas ágiles se utilizan en ambientes complejos para reducir dicha complejidad.

Framework Nexus

Consiste en roles, eventos, artefactos y técnicas que vinculan el trabajo de diferentes equipos.

- Tres a nueve equipos Scrum.
- Trabajan sobre un único Product Backlog.
- Tiene un único Product Owner al que entregar el producto.

Complejidad

Esta está dada por las siguientes dependencias entre los equipos:

- **Requerimientos:** Los alcances pueden superponerse.
 - La forma en la que se implemente cada requerimiento puede afectar a los demás requerimientos.
- **Conocimiento del dominio:** El conocimiento del sistema de negocio debería mapearse a los equipos Scrum para minimizar las interrupciones entre los mismos durante el Sprint.

Responsabilidades

Nexus Integration Team

- Responsable de asegurar que se produzca un Incremento Integrado al menos una vez por Sprint.
- La integración incluye abordar las restricciones técnicas y no técnicas del equipo multifuncional que pueden impedir la capacidad de un Nexus para

entregar constantemente un Incremento Integrado.

- Desarrollar una Definition of Done que se pueda aplicar en cada Sprint.

Product Owner (PO)

- Responsable de maximizar el valor del producto y el trabajo realizado e integrado por los Scrum Teams en el Nexus.
- Gestión del Product Backlog.

Scrum Master

- Responsable de asegurar que el marco de trabajo Nexus se entienda y promulgue su correcto funcionamiento.
- Puede ser Scrum Master en uno o más de los Scrum Teams en el Nexus.

Miembros del Nexus Integration Team

- Ayudan a los Scrum Teams a adoptar herramientas y prácticas que contribuyen a mejorar la capacidad de los Scrum Teams para entregar un Incremento Integrado útil cumpliendo la Definition of Done.

Eventos

Nexus agrega ciertos eventos a los ya definidos en Scrum.

- La duración de los eventos se basa en los definidos en Scrum.

Nexus Sprint

- Los Scrum Teams producen un único Integrated Increment.

Refinamiento entre equipos

- Busca reducir o eliminar las dependencias entre los equipos dentro del Nexus.
- El Product Backlog se descompone para que las dependencias sean transparentes. Para que se puedan identificar entre equipos y se eliminen o minimicen.
- Ayuda a los Scrum Teams a prever qué equipo entregará qué elementos del Product Backlog.
- Identifica dependencias entre los equipos.

- El refinamiento entre equipos es continuo.

Nexus Sprint Planning

- Busca coordinar las actividades de todos los Scrum Teams para un solo Sprint.
- Los representantes de cada Scrum Team y el PO se reúnen para planificar el Sprint.
- **Resultados:**
 - Objetivo de Sprint de cada Scrum Team.
 - Objetivo del Nexus.
 - Un Nexus Sprint Backlog.
 - Un Sprint Backlog.

Nexus Daily Scrum

- Se discuten problemas de integración y se inspecciona el progreso hacia el objetivo de Sprint del Nexus.
- Solo asisten representantes de cada Scrum Team.
- La Daily Scrum de cada Scrum Team complementa la Nexus Daily creando planes para el día, centrados principalmente en problemas de integración planteados en la Nexus Daily.
- Los Scrum Teams no sólo se reúnen o coordinan en el Nexus Daily Scrum. Durante el día puede existir comunicación entre equipos con mayor detalle sobre adaptar o replanificar el resto del trabajo del Sprint.

Nexus Sprint Review

- Se lleva a cabo al final del Sprint para brindar retroalimentación al Integrated Increment terminado durante el Sprint.
- Determina adaptaciones futuras.
- Reemplaza el Sprint Review de Scrum.

Nexus Sprint Retrospective

- Planificar formas de mejorar la calidad y eficacia en todo el Nexus.
- Inspecciona cómo fue el último Sprint para ver posibles mejoras.

- Las retrospectivas de los Scrum Teams complementan la Nexus Sprint Retrospective mediante el uso de inteligencia "de abajo hacia arriba" para centrarse en los problemas que afectan al Nexus en su conjunto.

Artefactos

Product Backlog

- Hay uno solo.
- Contiene la lista de todo lo que el Nexus y sus Scrum Teams necesitan para mejorar el producto.
- Debe entenderse a tal nivel que permita detectar y minimizar las dependencias.
- Responsabilidad del PO.

Nexus Sprint Backlog

- Compuesto por el objetivo del Sprint del Nexus y elementos del Product Backlog de cada Scrum Team.
- Se utiliza para resaltar las dependencias y el flujo de trabajo durante el Sprint.
- Su compromiso es el objetivo del Sprint del Nexus.
- Se crea en el Sprint Planning y se agrega al Sprint Backlog del Nexus.

Integrated Increment

- Es la suma actual de todo el trabajo integrado completado por un Nexus en relación con el objetivo del producto.
- Se inspecciona en la Nexus Sprint Review, pero puede entregarse antes de la misma.
- Debe cumplir con la Definition of Done.
- **Compromiso:** Definition of Done.
- El Incremento sólo termina cuando está integrado, es de valor y es utilizable.



Los Scrum Teams son equipos auto-organizados que pueden aplicar su propia Definition of Done, la cual debe ser al menos igual de estricta que la definida por el Nexus Integration Team.

Filosofía Lean

Principios Lean

1. **Eliminar desperdicios** → todo lo que no genera valor es desperdicio, se busca eliminar lo más que se pueda las cosas que no generan valor.
2. **Amplificar Aprendizaje** → Trasladar conocimiento individual a conocimiento organizacional.
3. **Embeber la Integridad conceptual** → atención continua a la excelencia técnica. Que la calidad del producto o servicio NO se negocia.
 - a. La calidad no se puede agregar al final. Es un hábito, una decisión incorporada al proceso de creación.
4. **Diferir compromisos** → El elemento más importante es la información para la toma de decisiones
 - a. Diferir compromisos hasta el último momento responsable. Se busca postergar la toma de decisiones hasta que tenga más información.
5. **Dar poder al equipo** → Personas autogestionadas, auto organizadas, en condiciones de hacerse cargo de asumir compromisos.
6. **Ver el todo** → Visualizar todo el conjunto
 - a. Intentar dar previsibilidad al objetivo final.
7. **Entregar lo antes posible** → entregas frecuentes.

Gastos en producción Lean (Dentro de desperdicios)

- Producción en exceso
- Stock
- Pasos extra en el proceso
- Búsqueda de Información

- Defectos
- Esperas
- Transportes



Tipos de Desperdicios

1. **Desperdicio necesario** → se busca minimizar porque no se puede eliminar del todo
 - a. Ejemplo: actividades que no están directamente relacionadas con la producción del producto pero si indirectamente. → Como la gestión sobre procesos.
2. **Puro desperdicio** → se busca evitar o eliminar

Desperdicios en Manufactura	Desperdicios en servicios basados en conocimiento
Inventario	Trabajo parcialmente terminado
Sobreproducción	Características extra
Proceso extra	Proceso extra
Transporte	Cambio de tareas Transferencia de conocimiento
Esperas	Esperas/Demoras
Movimiento	Cambio de contextos (Seteo)
Defectos	Defectos
Talento no utilizado	Talento no utilizado

Kanban

Método basado en la mejora continua, que busca gestionar la creación y entrega de productos.

Método para definir, gestionar y mejorar servicios que entregan trabajo del conocimiento, tales como servicios profesionales, trabajo o actividades en las que interviene la creatividad y el diseño tanto de productos de software como físicos

Se busca la visualización del trabajo (kanban = tarjetas de visualización)



NO gestiona proyecto, gestiona FLUJOS.

Valores de Kanban

- Transparencia
- Equilibrio
- Colaboración
- Foco en el cliente

- Flujo → marca la diferencia entre SCRUM y Kanban (en Scrum se trabaja en proyectos e iteraciones)
- Liderazgo
- Entendimiento
- Acuerdo
- Respeto

Principios de Kanban

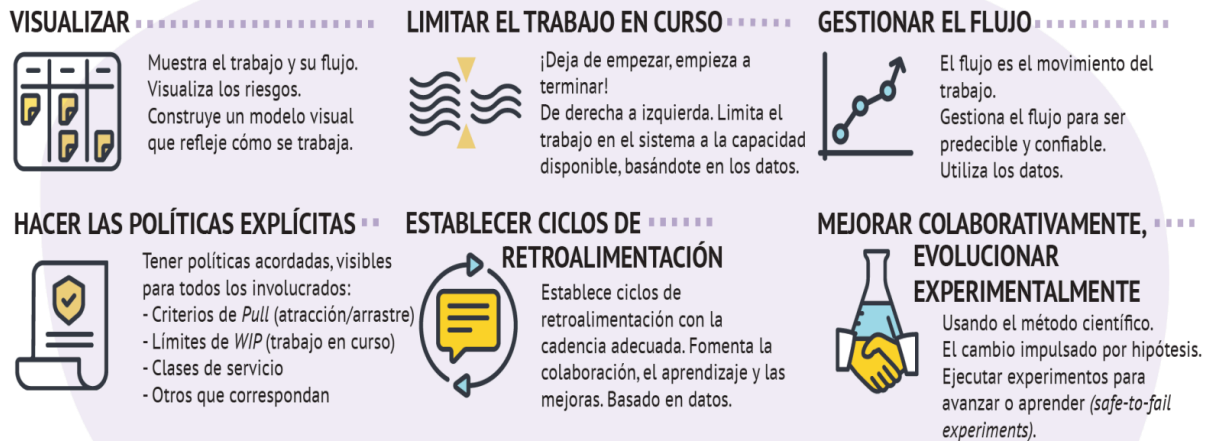
Gestión de Cambios

- Comenzar con lo que haces ahora (con el proceso actual) → mejora la resistencia al cambio
 - Entender los procesos actuales tal y como están siendo realizados en la actualidad
 - Respetar los roles actuales, las responsabilidades de cada persona y los puestos de trabajo.
 - Propone cambios evolucionarios y no revolucionarios
- Acordar la búsqueda de la mejora a través del cambio evolutivo
- Fomentar actos de liderazgo a todos los niveles

Entrega de Servicios

- Comprender y enfocarse en cumplir las necesidades y expectativas del cliente
- Gestionar el trabajo; dejar que los trabajadores se auto organicen en torno a él
- Revisar periódicamente la red de servicios y sus políticas para mejorar los resultados entregados.

Prácticas Generales de Kanban



Visualizar el trabajo

- Permite absorber y procesar una gran cantidad de información en un corto intervalo de tiempo.
- Habilita la cooperación, ya que todo el mundo tiene la misma imagen.
- Permite tomar decisiones de una manera más rápida y colaborativa.
- En el proceso actual, se busca generar colas (columnas) dentro del tablero. Con un orden FIFO.
- Se busca dividir el trabajo en piezas que permita visualizarlo, con colores distintos para cada tipo y en notas diferentes.



No hay fecha de fin ni inicio como si fuera un proyecto, ya que el tablero busca demostrar un FLUJO constante de trabajo.

Limitar el trabajo en curso

- Los sistemas eficaces son los que se centran más en el flujo de trabajo y menos en tener ocupados a los trabajadores.
- Cuando los recursos están ocupados completamente, no hay holgura en el sistema y como resultado el flujo es deficiente, como ocurre en una autopista en las horas picos.
- Limitar el WIP (trabajo en proceso) provoca conversación y mejora. → trabajo sobre la idea de que en las colas se puedan generar cuellos de botella.

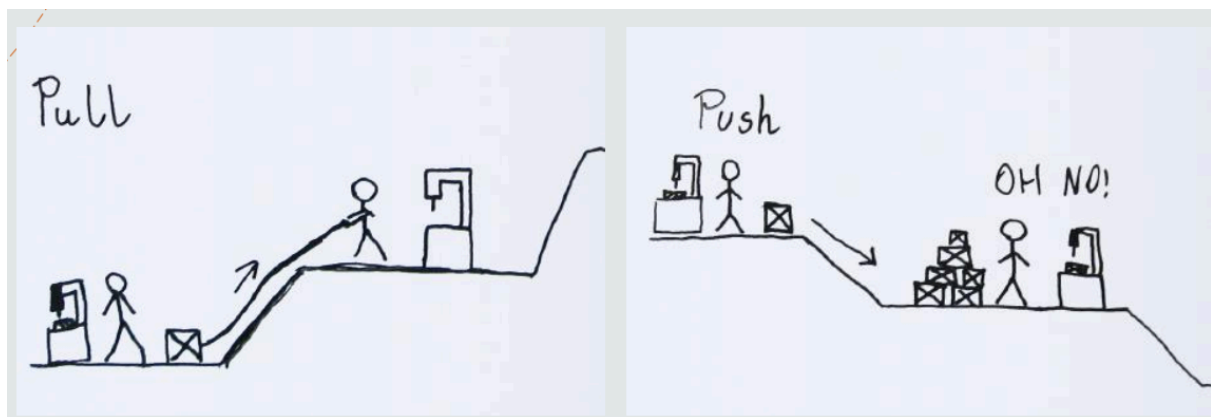
- Limitamos el WIP para equilibrar la ocupación y asegurar que haya flujo de trabajo.
- Asignar límites explícitos de cuántos ítems pueden haber en progreso en cada estado del flujo de trabajo por cola.

Administración de Colas

- Foco en el flujo "fin a fin"
- Flow = centrado en el cliente
- Mediante FIFO

Sistema de arrastre:

- Cuando se tiene disponibilidad, voy y agarro nuevo trabajo (PULL)
- "Deja de empezar y empieza a terminar" → se busca mirar los tableros o revisar de derecha a izquierda.
 - Que le hacen falta a las tareas para estar listas.
 - Ayuda a mejorar la administración de colas.



Gestionar el flujo de trabajo

- El objetivo es poder terminar el trabajo de la forma más fluida y predecible posible, mientras se mantiene un ritmo sostenido.
- Limitar el trabajo en curso nos ayuda a asegurar un flujo suave y predictivo.
- El seguimiento y medición del flujo da como resultado info. valiosa para gestionar expectativas de clientes, predicciones y mejoras.

Definir tipos de trabajos

- Requerimientos (casos de uso, historias de usuario, features, etc.)
- Defectos
- Desarrollo (Mantenimiento, refactorización, etc.)
- Solicitudes (de cambio, de mejora, etc.)

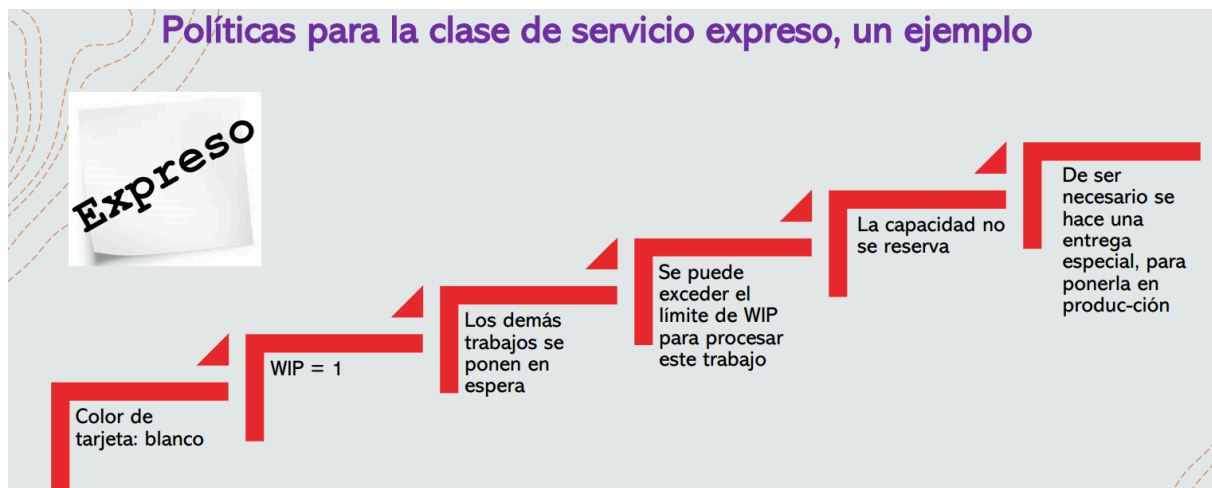
Asignando capacidad en función de la demanda

- Por ejemplo: 60% en req. 30% mantenimiento, 10% defectos

Hacer explícitas las políticas

- Todas las políticas deben ser acordadas entre todas las partes involucradas, incluyendo a los clientes, interesados y trabajadores responsables del trabajo que está en el tablero.
- Las políticas deben estar expuestas en un lugar destacado.
- A nivel de equipo, los acuerdos son una buena forma de introducir las políticas
- Deben ser:
 - Pocas
 - Sencillas
 - Bien definidas
 - Visibles
 - Aplicables en todo momento
 - Fácilmente modificables por los que prestan el servicio

Políticas Explícitas para cada clase de servicio



¿Como aplicar Kanban?

- **Proceso:** modelar nuestro proceso.
- **Trabajo :** decidir la unidad de trabajo.
- **Límites de WIP:** limitar el WIP para ayudar al flujo de trabajo.
- **Política:** definir políticas de calidad.
- **Cuellos de Botella y Flujo:** mover recursos a los cuellos de botella.
- **Clase de Servicio:** diferentes trabajos tienen diferentes políticas – definición de hecho ("done"), para cada estado.
- **Cadencia:** Releases, planificaciones, revisiones

Cola de Producto	Análisis		Desarrollo		Listo para Build	En Testing		En Producción
	En progreso	Hecho	En progreso	Hecho		En Progreso	Listo para Despliegue	
Casos de Uso 60 %								
Mantenimiento 30 %								
Defectos 10%								

Cadencias

Cadencia	Ejemplo de frecuencia	Propósito
Team Kanban Meeting	Diaria	Observar y seguir el estado y flujo del trabajo (no de los trabajadores). ¿Cómo podemos entregar los elementos de trabajo más rápido en el sistema? ¿Hay capacidad disponible? ¿Qué debemos tomar a continuación?
Team Retrospective	Quincenal o mensual	Reflexionar sobre cómo el equipo gestiona su trabajo y cómo pueden mejorar
Team Replenishment Meeting	Semanalmente o a demanda	Seleccionar los elementos de la lista de trabajo para realizar a continuación

Métricas de Software

El dominio de las métricas del software se divide en:

- Métricas de proceso
 - Buscan evaluar la salud de mi proceso
- Métricas de proyecto
 - Las métricas son fundamentales para poder “jugar” con la triple restricción en un proyecto
- Métricas de producto

Las métricas del proyecto se consolidan para crear métricas de proceso que sean públicas para toda la organización del software

Métricas básicas para un proyecto de software

En una gestión tradicional de un proyecto

1. Tamaño del producto

- a. Cantidad de casos de uso, user stories, cantidad de líneas de código.

2. Esfuerzo

- a. Horas lineales en una determinada actividad

3. Tiempo (Calendario)

- a. Cantidad de recursos disponibles para ejecutar actividades
- b. Que actividades en paralelo y cuales secuenciales

4. Defectos

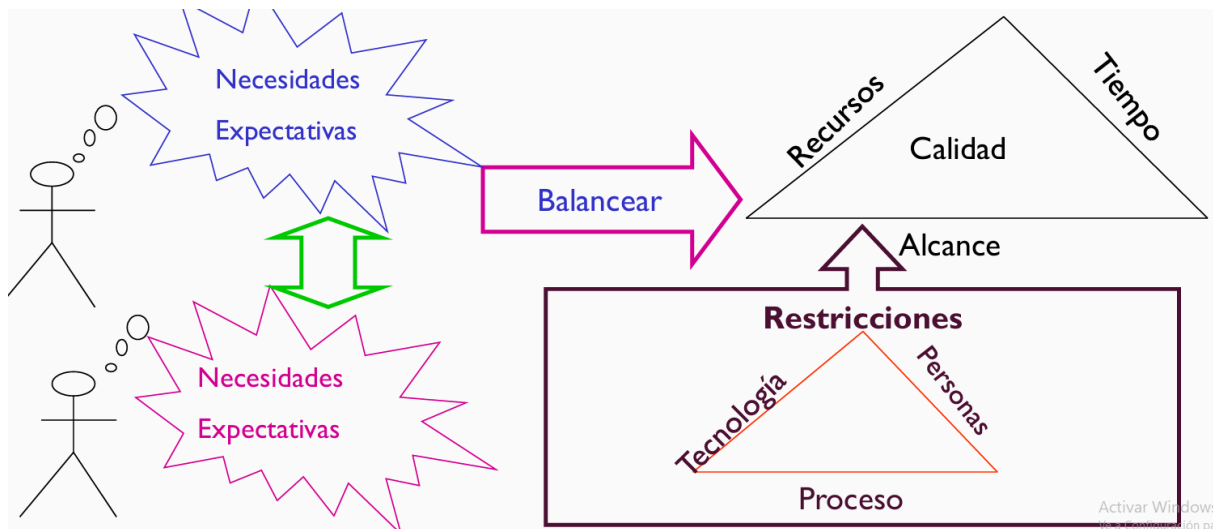
Ejemplos de Métricas Tradicionales

- Las métricas NUNCA deben servir para medir a las personas
- Tomar las métricas llevar un esfuerzo, por ende queremos que la métrica se calcule "sola" y no destinar mucho esfuerzo en obtenerlas.

Desarrollador	Equipo de Desarrollo
1. Esfuerzo 2. Esfuerzo y duración estimada y actual de una tarea. 3. % de cobertura por el unit test 4. Número y tipo de defectos encontrados en el unit test. 5. Número y tipo de defectos encontrados en revisión por pares.	1. Tamaño del producto 2. Duración estimada y actual entre los hitos más importantes. 3. Niveles de staffing actuales y estimados. 4. Nro. de tareas planificadas y completadas. 5. Distribución del esfuerzo 6. Status de requerimientos. 7. Volatilidad de requerimientos. 8. Nro. de defectos encontrados en la integración y prueba de sistemas. 9. Nro. de defectos encontrados en peer reviews. 10. Status de distribución de defectos. 11. % de test ejecutados
Organización	
1. Tiempo Calendario 2. Performance actual y planificada de esfuerzo. 3. Performance actual y planificada de presupuesto 4. Precisión de estimaciones en Schedule y esfuerzo 5. Defectos en Release	

Características de Métricas Tradicionales

- **Periodicidad:** en un proyecto con un horizonte largo, no tiene sentido tener métricas muy frecuentes. Se debe buscar una buena periodicidad.
 - Si la métrica siempre me da lo mismo, es porque no nos da un valor agregado.
 - Un ítem clave o complejo es conseguir definir métricas que nos den valor agregado
- Nos permiten balancear la triple restricción.



Métricas en Ambientes Ágiles

Para procesos empíricos

La medición es una SALIDA, no una actividad

La filosofía minimalista sobre las métricas en agile:

- Medir lo que sea necesario y nada más.

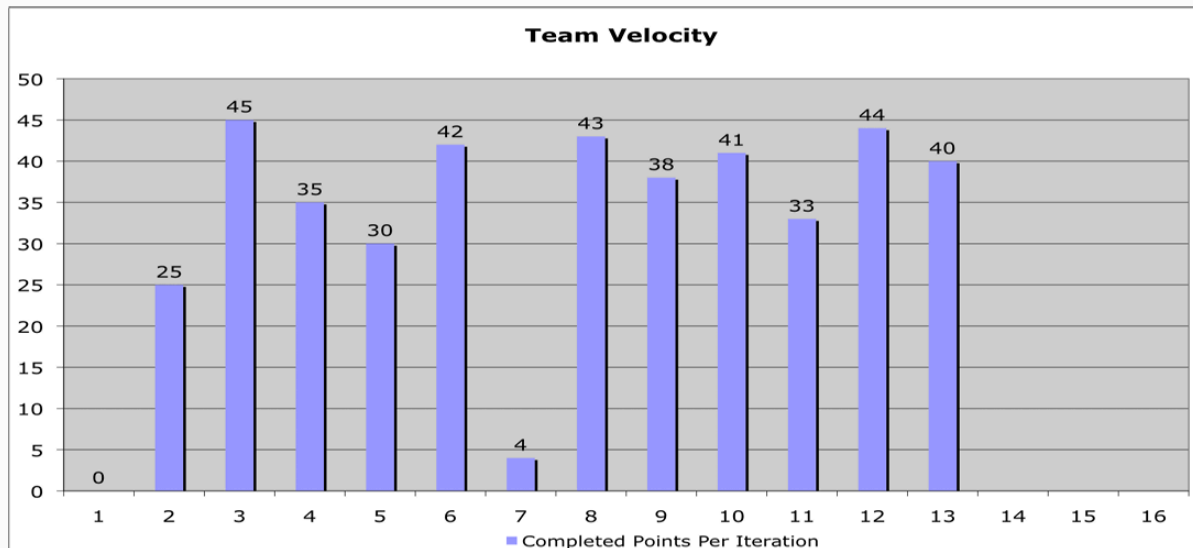
Existen 2 principios ágiles fundamentales:

1. Nuestra mayor prioridad es satisfacer al cliente por medio de entregas tempranas y continuas de software valioso, funcionando
2. El Software funcionando es la principal medida de progreso.

Velocidad

La métrica fundamental acá es la **"VELOCIDAD"**, los story points quemados y aceptados por el PO.

- En la Sprint Review se mide la velocidad, porque se revisa la entrega y se acepta, contando los puntos de historia.



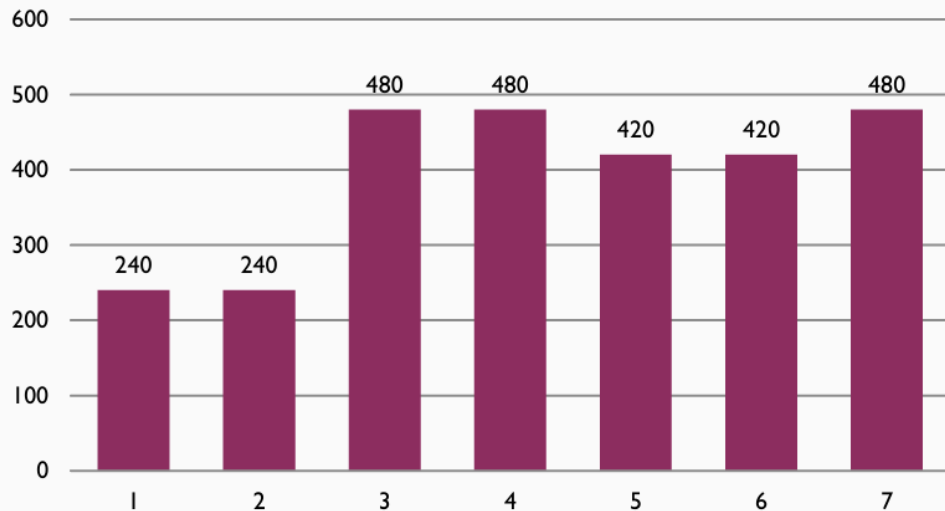
Capacidad

Sirve para estimar cuantos puntos de historia voy a ser capaz de quemar en el próximo sprint.

- Pero si no se tiene esa información todavía, se toman en base a horas.
- Utilizamos la descomposición de tareas del sprint backlog para trabajar las tareas en términos de horas estimadas. (No es lo ideal, pero en primeras iteraciones no es posible medir en story points tan facilmente)
- Se calcula en la planning para saber cuanto trabajo tomar en el mismo.

Sprint	1	2	3	4	5	6	7	Total
Horas	240	240	480	480	420	420	480	2760
Puntos de Historia	30	30	45	60	58	52	60	335

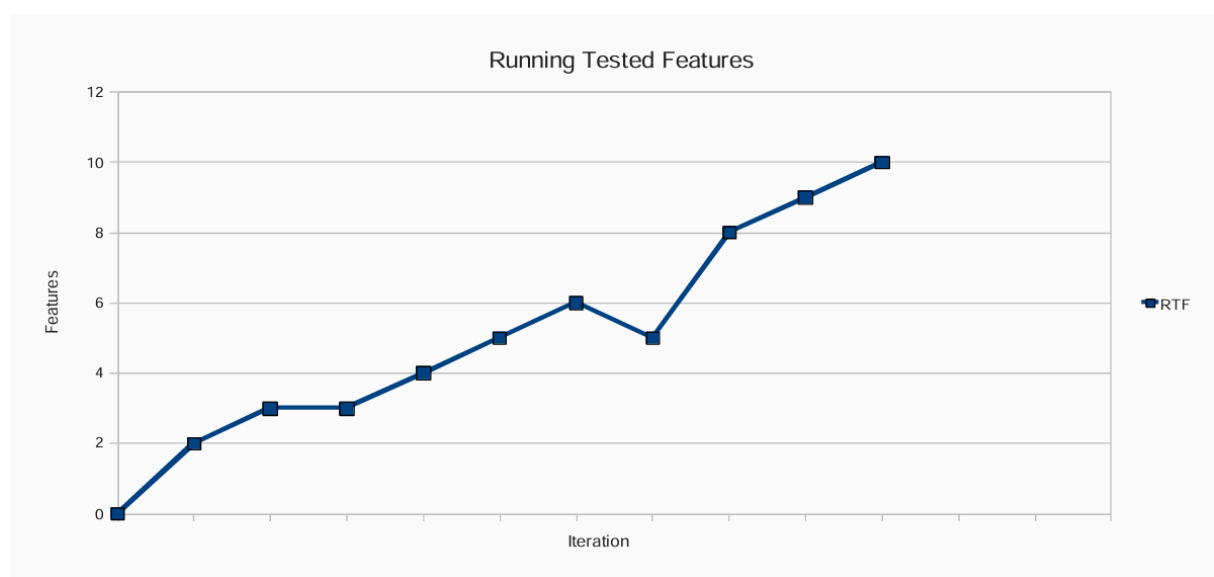
Capacidad



Running Tested Features (RTF)

Podemos observar una medida de progreso, pero no nos determina cuanto es el software funcionando que estamos entregando.

- Por eso no es tan utilizada.
- Se calcula al final de cada iteración en la Sprint Review, según las características probadas y aceptadas por el PO.



Métricas en Kanban

Trabaja con procesos empíricos pero con enfoque LEAN



SCRUM tiene que ver más con la gestión de proyecto, pero KANBAN sirve más para la gestión de un proceso continuo (Flujo).

Lead Time

Es la vista del cliente

- Tiempo desde que el usuario solicita una característica y es entregada efectivamente
- Es la métrica que registra el tiempo que sucede entre el momento en el cual se está pidiendo un ítem de trabajo y el momento de su entrega (el final del proceso). Se suele medir en días de trabajo.
- Ritmo de entrega

Cycle Time

Interno para el equipo

- Empieza a contar desde que se empieza a trabajar efectivamente en una tarjeta hasta terminarla.
- Es la métrica que registra el tiempo que sucede entre el inicio y el final del proceso, para un ítem de trabajo dado.
- Se suele medir en días de trabajo o esfuerzo
- Medición más mecánica de la capacidad del proceso
- Ritmo de Terminación

Touch Time (Tiempo de Tocado)

- El tiempo en el cual un ítem de trabajo fue realmente trabajado (o "tocado") por el equipo.
- Cuántos días hábiles pasó este ítem en columnas de "trabajo en curso", en oposición con columnas de cola / buffer y estado bloqueado o sin trabajo del equipo sobre el mismo.

Eficiencia del Ciclo de Proceso

% Eficiencia ciclo proceso = Touch Time / Lead Time

- A más cerca está el Lead Time del Touch Time, quiere decir que soy más eficiente, en cuanto al trabajo que fluye y tomar una pieza de trabajo que llega.

Métricas Orientadas a Servicio

En Kanban buscamos medir en base al servicio que brindamos.

Nos permite gestionar en el servicio que ofrecemos:

- Expectativa de nivel de servicio que los clientes esperan
- Capacidad del nivel de servicio al que el sistema puede entregar.
- Acuerdo de nivel de servicio que es acordado con el cliente.
- Umbral de la adecuación del servicio el nivel por debajo del cual este es inaceptable para el cliente.

Métricas en cada Enfoque

■ Tradicionales

- Esfuerzo
- Tiempo
- Costos
- Defectos

■ Ágiles

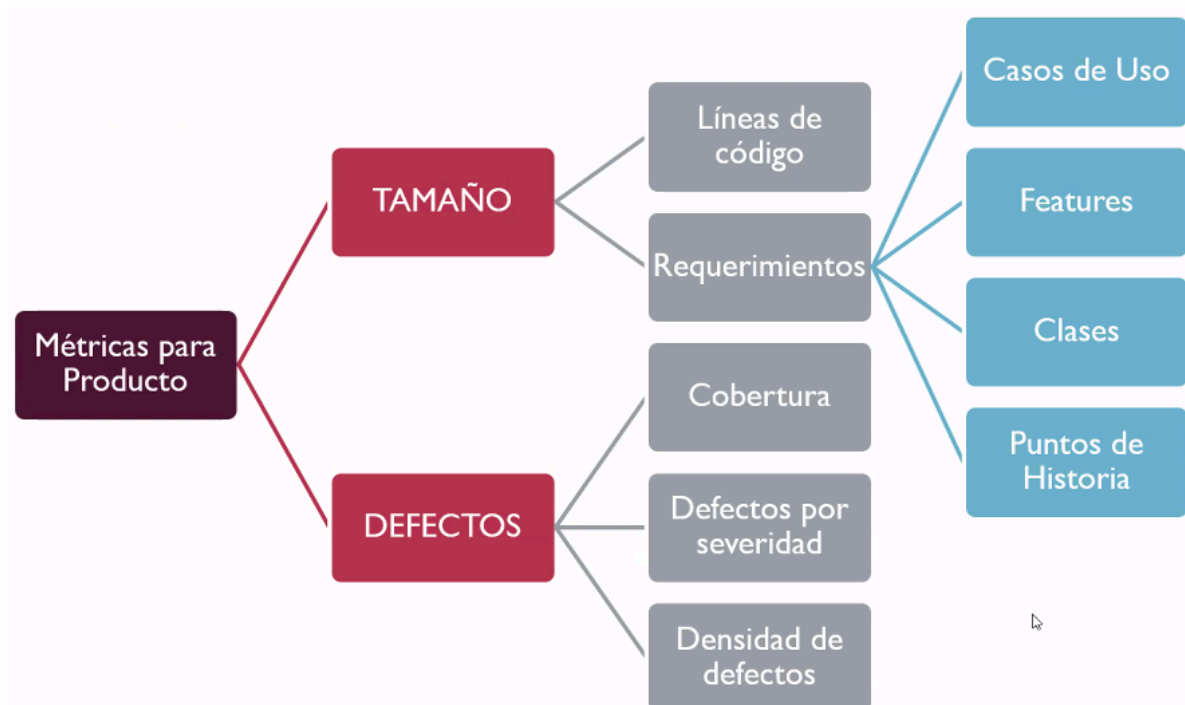
- Velocidad
- Capacidad
- Running Tested Features

■ Lean

- Lead Time
- Cycle Time
- Touch Time
- Eficiencia Proceso

- Costo no es una métrica básica, pero se utiliza bastante en la gestión tradicional.

Métricas del Producto de Software



Conceptos transversales a todas las métricas

- Las métricas no sirven para medir personas o utilidad. Es un mal uso del concepto
- Nos sirve para mejorar la forma en la que trabajamos
 - No importa el framework o contexto.

Prácticas Continuas

Continuous Integration (Integración Continua)

Práctica de desarrollo que promueve que los desarrolladores adopten la costumbre de integrar su código a un repositorio compartido varias veces al día.

- Cada integración luego es verificada por una serie de pruebas automatizadas, permitiendo al equipo detectar problemas de manera temprana.
- Busca asegurar que el software pueda ser desplegado en cualquier momento. O sea, que compile y sea de calidad.
- Cada desarrollador en su entorno de trabajo realiza pruebas unitarias, desarrollando con TDD.

- Al finalizar el desarrollo del componente y haber sido probado, sube el componente al repositorio de integración.



Continuous Delivery (Entrega Continua)

Disciplina de desarrollo de software en la que el software se construye de tal manera que puede ser liberado en producción en cualquier momento.

- Suma la automatización de pruebas de aceptación.
- No implica que se libere cada vez que hay un cambio, sino que el PO puede decidir en cualquier momento cuándo desplegarlo.
- La Integración Continua es un prerequisite para la Entrega Continua.
- Hay que asegurarse de tener un despliegue automatizado.
- El software siempre debe estar en estado de "entregable".



Continuous Deployment (Despliegue Continuo)

Consiste en desplegar el producto de forma automatizada.

- A diferencia de la Entrega Continua, acá no existe la intervención humana para desplegar el producto en producción.
- Se utilizan *pipelines* (tuberías) que contienen una serie de pasos que deben ejecutarse en un orden determinado para que la instalación sea satisfactoria.



Canary Deployment

Práctica de realizar lanzamientos por etapas.

- Primero se implementan los cambios para una pequeña parte de los usuarios para ver su validación.
- Al tener la aceptación de los usuarios, se extiende al resto de los mismos.

Blue-Green Deployment

Modelo de lanzamiento de aplicaciones que transfiere gradualmente el tráfico de usuarios de una versión anterior de una aplicación o microservicio a una nueva versión casi idéntica.

- Versión anterior → **entorno azul**.
 - Nueva versión → **entorno verde**.
 - Al transferir todo el tráfico del entorno azul al verde, el azul puede quedar en espera en caso de reversión del cambio.
-

Calidad de Producto

La calidad del software se puede enfocar en el Producto, el Proceso y el Proyecto.

Técnicas y Herramientas

- **Para el Proyecto:** Revisiones Técnicas y Auditorías.
- **Para el Producto:**
 - **Aseguramiento de Calidad:** Revisiones Técnicas y Auditorías (De Configuración).
 - **Control de Calidad:** Testing.

Revisiones de Calidad:

- Son el principal método para validar la calidad de un proceso o producto.
- Un grupo examina el producto o proceso para encontrar problemas.
- **Tipos:** Inspecciones (remover defectos), Revisiones de progreso, Revisiones de Calidad (producto y estándares).

Puntos Claves

- La calidad del software se puede analizar desde la perspectiva del **proceso** y del **producto**.

- La calidad del software es difícil de medir.
 - El **software como proceso** es el fundamento para mejorar la calidad.
 - **"Trabajar con calidad es más barato que hacerlo sin calidad"**.
 - La mejora de procesos ha demostrado retornos de inversión sustanciales.
-

Testing en Ambientes Ágiles

Manifiesto del Testing Ágil

El testing ágil se basa en un cambio de mentalidad, resumido en 5 valores clave:

1. Pruebas "durante" sobre pruebas "al final"

- En lugar de ser una fase final, el testing es una actividad continua que ocurre en paralelo al desarrollo.

2. Prevenir defectos sobre encontrar defectos

- El enfoque se mueve de la simple detección de bugs (encontrar) a un involucramiento temprano para evitar que se introduzcan (prevenir).

3. Entender lo que se está probando sobre verificar la funcionalidad

- No se trata solo de marcar ítems en una checklist ("checker"), sino de comprender el propósito del negocio y las necesidades del usuario ("tester").

4. Construir un mejor sistema sobre romper el sistema

- El objetivo del tester no es solo "romper" el software, sino colaborar activamente para ayudar a construir el mejor producto posible.

5. Responsabilidad de todo el equipo por la calidad sobre el tester es responsable de la calidad

- La calidad deja de ser la responsabilidad de una sola persona o un departamento (que actúa como "Quality Gate") y se convierte en una responsabilidad compartida por todo el equipo (desarrolladores, POs, testers).

Enfoque Tradicional vs. Enfoque Ágil

Enfoque Tradicional (Cascada)

- **Secuencial:** Las fases (Requirements, Design, Development, Test, Deploy) ocurren una después de la otra.
- **Testing como Fase:** El testing es una fase separada que ocurre *después* de que el desarrollo ha terminado.
- **Ciclos Largos:** Los ciclos de feedback son muy largos (meses o años).

Enfoque Ágil

- **Iterativo e Incremental:** El ciclo (Define, Analyse, Design, Test, Deploy) se repite en iteraciones cortas.
- **Testing como Actividad:** El testing es una actividad integrada *durante* toda la iteración.
- **Ciclos Cortos:** El feedback es rápido y constante (días o semanas).

Principios y Prácticas del Testing Ágil

9 Principios Clave

1. Testing se mueve hacia adelante en el proyecto.
2. Testing no es una fase.
3. Todos hacen testing.
4. Reducir la latencia (el tiempo de espera) del feedback.
5. Las pruebas representan expectativas (criterios de aceptación).
6. Mantener el código limpio, corregir los defectos rápido.
7. Reducir la sobrecarga de documentación de las pruebas.
8. Las pruebas son parte del "Done" (Definición de Hecho).
9. Se pasa de "probar al final" a un enfoque "Conducido por Pruebas".

6 Prácticas Concretas

1. **TDD (Test Driven Development):** Desarrollo conducido por pruebas unitarias.
2. **ATDD (Acceptance Test Driven Development):** Desarrollo conducido por pruebas de aceptación.

3. Pruebas de unidad e integración automatizadas.
4. Pruebas de Regresión a nivel de sistema automatizadas.
5. **Pruebas exploratorias:** Pruebas manuales creativas y de aprendizaje para descubrir lo inesperado.
6. Control de versión de las pruebas junto con el código.

ATDD (Desarrollo Conducido por Pruebas de Aceptación)

Es un ciclo de colaboración:

- Inicia con una **Característica / Feature** (con ejemplos).
- Se define una **Historia de Usuario**.
- Se generan **Pruebas de Aceptación de alto nivel** (explorando ejemplos).
- Estas pruebas guían la **Corrección de Defectos** y la **Automatización de las pruebas**.
- Se realizan **Pruebas Exploratorias** para validar.
- Finalmente, se logra la **Aceptación de la Historia**.

El Rol del Tester Ágil

El tester ágil es un colaborador activo que ayuda a:

- **Product Owner (PO):**
 - En la preparación y revisión de historias.
 - En la definición de Criterios de Aceptación.
 - A entender mejor el negocio.
- **Developers (Desarrolladores):**
 - En la definición de estrategias de prueba.
 - En la automatización de pruebas (ATDD).
 - Dando feedback temprano sobre las historias.
 - Sugiriendo mejoras de usabilidad.
- **System Team (Equipo de Sistema):**
 - En la preparación y mantenimiento del Ambiente de Pruebas.

- Validando el "Done" de las historias.
- En la aceptación final de la historia con el PO.
- Generando métricas de calidad.

Cuadrantes del Testing Ágil

Es un modelo para clasificar las pruebas y asegurar una cobertura completa. Se divide en 4 cuadrantes:

- **Q1 (Orientado a Tecnología / Apoya al Equipo): Pruebas Automatizadas.**
 - Pruebas Unitarias (Unit Tests).
 - Pruebas de Componentes.
- **Q2 (Orientado a Negocio / Apoya al Equipo): Pruebas Automatizadas y Manuales.**
 - Pruebas Funcionales.
 - Pruebas de Historias (Story Tests).
 - Prototipos y simulaciones.
- **Q3 (Orientado a Negocio / Crítica al Producto): Pruebas Manuales.**
 - Pruebas Exploratorias.
 - Pruebas de Usabilidad.
 - UAT (User Acceptance Testing).
 - Pruebas Alpha / Beta.
- **Q4 (Orientado a Tecnología / Crítica al Producto): Pruebas con Herramientas (Tools).**
 - Pruebas de Performance y Carga.
 - Pruebas de Seguridad.
 - Pruebas de Disponibilidad, Escalabilidad, etc.

La Pirámide del Testing

Pirámide Tradicional (Anti-patrón "Cono de Helado")

- **Foco:** Búsqueda de bugs.

- **Base pequeña:** Pocas Pruebas Unitarias (1-5%).
- **Medio:** Pruebas de Integración (5-15%).
- **Cima grande:** Muchas Pruebas de GUI / End-to-End (80-90%).
- **Problema:** Lentas, frágiles y costosas de mantener.

Pirámide Ágil

- **Foco:** Prevención de bugs.
- **Base grande:** Muchas **Pruebas Unitarias** (80-90%). Son rápidas y estables.
- **Medio:** Pruebas de Servicio / Aceptación / API (5-15%).
- **Cima pequeña:** Pocas **Pruebas de GUI** (1-5%). Solo para flujos críticos.
- **Capa superior:** Pruebas Manuales / Exploratorias.

Automatización del Testing

La automatización es clave en agilidad y ofrece múltiples beneficios:

- Ejecución rápida.
 - Reusabilidad de los scripts.
 - Mayor cobertura de pruebas.
 - Mayor precisión (reduce el error humano).
 - Mayor alcance (permite pruebas difíciles de hacer manualmente).
 - Reducción de costos a largo plazo.
-

Testing de Software

Conceptos Fundamentales de Testing

¿Qué es el Testing?

- Es una actividad que intenta detectar defectos en el software, los cuales se asumen como presentes siempre.

- El objetivo es identificar defectos. Un testing exitoso es aquel que encuentra defectos (cuya presencia se asume en el software).
- Es una actividad de control; se realiza una comparación contra algo (requerimientos, especificaciones). Requiere el nivel de detalle más fino posible.
- El testing no asegura la calidad, sino que evidencia defectos. Es imposible asegurar la calidad total mediante testing.

¿Cuánto testing es suficiente?

- El testing exhaustivo (probar todas las combinaciones posibles) es impracticable.
- Un sistema de 20 pantallas, con menús, opciones y campos, podría requerir 240,000 casos de prueba.
- Si cada prueba tomara 10 minutos, probarlo todo llevaría 13.7 años.
- Por esto, se deben usar **métodos y estrategias** para seleccionar la menor cantidad de pruebas que cubran la mayor cantidad de funcionalidades.

Psicología del Testing

- Existe una psicología particular en el testing, especialmente en la relación entre Desarrolladores y Testers.

Error vs. Defecto vs. Falla

- **Error:** Se detecta y se corrige en la misma etapa donde se introdujo (ej. se detecta en Diseño y se corrige en Diseño).
- **Defecto:** Se traslada de etapa (ej. se introduce en Implementación y se encuentra en Pruebas).
- **Falla:** Mal funcionamiento del sistema visible por el usuario.

Proceso del Testing

Testing Metodológico

Se basa en un proceso estructurado.

1. Planificación y Control

2. Análisis y Diseño

- **Artefactos:**

- **Plan de Prueba:** Documento que guía el esfuerzo de testing.
- **Caso de Prueba:** Se crean o diseñan a partir de los escenarios identificados en las *user stories*. Es un paso a paso detallado que describe un escenario específico.
- **Contenido del Caso de Prueba:**
 - Seteo (datos específicos a ingresar, pre-condiciones).
 - Escenario (paso a paso para el escenario).
 - Resultado esperado (qué debe pasar).

3. Ejecución

- Implica correr los casos de prueba diseñados.

4. Evaluación y Reportes ("Seguimiento")

- **Reporte de Defectos:** Documentar los defectos encontrados.

Ciclo de Ejecución

- Contempla los pasos de **Ejecución y Seguimiento (o Evaluación)**. Esto representa el 50% del proceso.
- El otro 50% es la **Planificación y Diseño**, que NO forma parte del ciclo de ejecución.

Testing Ad Hoc

- Es un testing de "fuerza bruta" o sin un plan.
- Su principal problema es que es muy difícil reproducir un defecto encontrado, ya que no se siguieron pasos predefinidos.

Niveles de Prueba y Entornos

Niveles de Prueba

1. **Prueba Unitaria:** Hechas por el desarrollador.
2. **Prueba de Integración:** Se enfoca en la interacción entre componentes o módulos. La realiza Testing.

3. **Pruebas de Sistema:** Se prueba el sistema como un todo, de forma integral. La realiza Testing.
4. **Pruebas UAT (Aceptación de Usuario):** Se valida que el software cumple con las necesidades del usuario/cliente. Se llevan a cabo en el Despliegue.

Ambientes / Entornos

- **Desarrollo:** Donde los desarrolladores realizan las pruebas unitarias.
- **Prueba (Testing / QA):** Donde se realizan pruebas de integración y pruebas de Sistema.
- **Pre-Producción (Staging):** Donde se realizan las pruebas de UAT.
- **Producción:** El ambiente real del cliente.

Estrategias de Caja Negra (Black Box)

Se centran en probar la funcionalidad **basándose en las especificaciones**, sin mirar el código interno.

Estas se dividen en dos grandes grupos:

1. Técnicas Basadas en Especificaciones

A. Partición de Equivalencias

- **Concepto:** Divide todos los datos de entrada posibles en "clases de equivalencia". Se asume que si un valor de una clase funciona, todos los valores de esa clase funcionarán.
- **Proceso (2 Pasos):**
 1. **Identificar las clases de equivalencia** (Válidas y No Válidas).
 2. **Identificar los casos de prueba:** Se crea un caso de prueba por cada clase identificada.
- **Tipos de Clases a Identificar:**
 - Rango de valores continuos (ej. edad 18-65).
 - Valores Discretos (ej. 0, 1, 2, 3, 4, 5).
 - Selección simple (ej. "Rojo", "Azul", "Verde").
 - Selección múltiple (ej. tildar varias opciones).

- **Ejemplo (Impuestos):**

1. Hasta \ \$4000 (sin impuestos).
 2. Siguiendo \ \$1500 (10%).
 3. Siguiendo \ \$2000 (22%).
 4. Superior (40%).
- *Clases Válidas:* 0-4000, 4000.01-5500, 5500.01-7500, >7500.01.
 - *Clases No Válidas:* < 0, valores no numéricos.

B. Análisis de Valores Límites (BVA)

- **Concepto:** Es una variante de la Partición de Equivalencias. En lugar de tomar *cualquier* valor de la clase, se seleccionan los **bordes (límites)** de cada clase.
- **Razón:** La experiencia demuestra que los errores ocurren con mayor frecuencia en los límites de las clases.
- **Ejemplo (Impuestos):**
 - *Clase 1 (0-4000):* Probar 0, 4000.
 - *Clase 2 (4000.01-5500):* Probar 4000.01, 5500.
 - *Clase 3 (5500.01-7500):* Probar 5500.01, 7500.
 - *Límites inválidos:* -0.01 (si aplica), etc.

2. Técnicas Basadas en la Experiencia

A. Adivinanza de Defectos (Error Guessing)

- **Concepto:** El tester usa su experiencia, intuición y conocimiento del sistema para "adivinar" dónde es probable que haya errores.
- Ej. Probar campos vacíos, ingresar texto en campos numéricos, hacer clic muy rápido, etc.

B. Testing Exploratorio

- **Concepto:** Es un testing simultáneo de aprendizaje, diseño de pruebas y ejecución.

- No se parte de casos de prueba detallados, sino de una "misión" o "charter". El tester explora la aplicación y va documentando lo que encuentra.

Tipos de Pruebas (Funcional y No Funcional)

A Nivel Funcional

- **Requerimientos de Negocio**
- **Funcionalidad:**
 - Operación normal (camino felices).
 - Tests Negativos (qué pasa si falla).
- **Documentación:** (Ej. Manual de Usuario).

A Nivel No Funcional

- **Prueba de Estrés:** Utilizar a la máxima capacidad comprometida (y superarla) para ver cómo responde el hardware/software.
- **Prueba de Escala Completa:** Se lleva a la máxima capacidad, pero sin exigir más.
- **Pruebas de Interfaces**
- **Pruebas de Seguridad**
- **Pruebas de Volumen de Datos**

Otros Tipos

- **Smoke Test:** Prueba rápida y superficial para verificar que las funcionalidades principales del sistema funcionan (ej. "enciende"). Se hace antes de aceptar una nueva versión para pruebas más profundas.
- **Exploratorias:** (Ver sección de Caja Negra).

Gestión de Defectos (Severidad vs. Prioridad)

Severidad de Defectos

- **Lo define el equipo de Testing.**
- Mide el impacto técnico del defecto en el sistema.

- **Bloqueante:** Impide continuar con las pruebas.
- **Grave:** Funcionalidad principal no opera.
- **Leve:** Funcionalidad secundaria no opera.
- **Cosmética:** Error visual, de texto, etc.

Prioridad de Defectos

- **Lo define el Cliente (o Product Owner en Scrum).**
- Mide la urgencia de negocio para corregir el defecto.
 - **Urgente**
 - **Media**
 - **Baja**

El equipo de desarrollo priorizará qué corregir basándose en una combinación de la Severidad (definida por QA) y la Prioridad (definida por el Negocio).

Conceptos Extra

- **Cobertura:** Porcentaje de posibles escenarios cubiertos sobre el total de escenarios. Dado que el testing exhaustivo es imposible, la cobertura total (100%) es una utopía.
- **Mejor Método:** El mejor método no es usar una sola técnica. Usar una **variedad de técnicas** (Particiones, BVA, Exploratorio, etc.) ayudará a un testing más efectivo y eficiente.

Revisiones Técnicas

Verificación y Validación (V&V)

La V&V es un proceso que dura todo el ciclo de vida, desde la revisión de requisitos hasta las pruebas.

- **Validación:** ¿Estamos construyendo el **producto correcto**? (Se compara con el "Mundo Real" o las necesidades del cliente).

- **Verificación:** ¿Estamos construyendo el producto **correctamente**? (Se compara contra la especificación o "Definición del Problema").

Costo de las Fallas

- El costo de reparar una falla (defecto) crece exponencialmente a medida que avanza el proyecto.
- Una falla que cuesta '1' en Requerimientos, puede costar '10' en Prueba de Integración y '100' después del Release (lanzamiento).
- El "retrabajo evitable" puede consumir entre el 40-50% del costo total de desarrollo.

Error vs. Defecto (Definición de Inspección)

- **Falla:** Es el término general para un error en un producto de trabajo.
- **Error:** Se origina en el producto de trabajo que se está inspeccionando actualmente.
- **Defecto:** Se origina en un producto de trabajo *predecesor* (ej. un error en los Requerimientos que se encuentra en el Diseño).

Las fallas se clasifican en:

- **Mayor:** Puede causar una falla operacional o un resultado inesperado.
- **Menor:** Una condición no deseable (ej. violación de estándares) que no causa una falla operacional, pero puede afectar el mantenimiento.
- **Cosmético:** Errores de tipeo, ortografía o gramática.

Revisiones Técnicas

Las Revisiones Técnicas son un proceso de **V&V estático** (no se ejecuta el código). Su objetivo principal es detectar y corregir defectos en etapas tempranas.

- **Ventajas:**
 - Se pueden descubrir muchos errores.
 - Se pueden inspeccionar versiones incompletas del producto.
 - Se pueden considerar otros atributos de calidad (mantenibilidad, claridad).

- **Desventajas:**
 - Requieren tiempo para organizarse.
 - Pueden parecer que ralentizan el proceso al inicio.
 - Implican costos de entrenamiento e infraestructura.

Tipos de Revisiones

Existen múltiples tipos, pero se destacan dos principales:

Walkthroughs (Recorridas)

- **Objetivo:** Capacitar desarrolladores, obtener un retorno rápido.
- **Atributos:**
 - Proceso **informal**.
 - Poca o ninguna preparación.
 - No se suelen tomar métricas.
 - El autor dirige la sesión y los participantes hacen preguntas.

Inspecciones

- **Objetivo:** Detectar y remover todos los defectos de forma eficiente.
- **Atributos:**
 - Proceso **formal** y controlado.
 - Requiere preparación individual.
 - Se usan checklists.
 - Se miden métricas (ej. densidad de defectos, esfuerzo por defecto).

El Proceso de Inspección Formal

La inspección es una actividad clave de garantía de calidad.

- **Qué SÍ es:** La forma más barata y efectiva de encontrar fallas, una forma de proveer métricas, promover el trabajo en grupo y mejorar la calidad.
- **Qué NO es:** Una reunión para encontrar soluciones, una evaluación del desempeño del autor, o una reunión para aprobar un producto.

Roles Clave

- **Autor:** Creador del producto a inspeccionar.
- **Moderador:** Lidera y planifica la revisión (rol clave).
- **Anotador:** Registra los hallazgos.
- **Lector:** Lee el producto durante la reunión.
- **Inspector:** Examina el producto antes de la reunión para encontrar defectos.

Pasos del Proceso de Inspección

1. **Planificación:** Elegir equipo, preparar material y calendario.
2. **Visión General:** (Opcional) Presentar el proceso y el producto.
3. **Preparación:** Análisis **individual** para encontrar potenciales defectos (crucial).
4. **Reunión de Inspección:** Análisis en equipo para recolectar defectos. (Máximo 2 horas).
5. **Corrección:** El autor corrige los defectos encontrados.
6. **Seguimiento:** El moderador verifica las correcciones y recolecta los datos (métricas).

Puntos Clave para una Buena Revisión

- **Revisar al producto, no al productor.**
- Fijar una agenda y cumplirla.
- Limitar el debate: solo se enuncian los problemas, no se buscan soluciones en la reunión.
- Tomar notas escritas.
- Limitar el número de participantes e insistir en la preparación previa.